



C\$50 Finance

Problem set



یک وبسایت پیاده‌سازی کنید که از طریق آن کاربران بتوانند سهام را بخرند و بفروشند.

پیش‌زمینه

اگر نمی‌دانید خرید و فروش سهام (سهام‌های یک شرکت) به چه معناست، برای آشنایی [اینجا](#) را مطالعه کنید.

شما در حال پیاده‌سازی C\$50 Finance هستید؛ یک وب اپلیکیشن که از طریق آن می‌توانید سبدهای سهام را مدیریت کنید. این ابزار نه تنها به شما امکان بررسی قیمت واقعی سهام و ارزش سبد سهام را می‌دهد، بلکه امکان خرید و فروش سهام را به واسطه کوئری زدن برای قیمت آن هم می‌دهد.

ابزارهایی وجود دارند (یکی از آنها به نام IEX شناخته می‌شود) که به شما امکان می‌دهند قیمت سهام را از طریق API (application programming interface) آنها با استفاده از این [آدرس](#) دانلود کنید. توجه داشته باشید که چگونه نماد نتفلیکس (NFLX) در این URL تعبیه شده است؛ IEX اینگونه می‌داند اطلاعات چه کسی را برگرداند. این لینک در واقع داده‌ای برمی‌گرداند چرا که IEX نیازمند استفاده از یک کلید API است. اگر داده‌ای برگرداند، پاسخی در قالب JSON (JavaScript Object Notation) خواهید داشت. به نمونه‌ی زیر توجه کنید:

```
{
  "avgTotalVolume":6787785,
  "calculationPrice":"tops",
  "change":1.46,
  "changePercent":0.00336,
  "close":null,
  "closeSource":"official",
  "closeTime":null,
  "companyName":"Netflix Inc.",
  "currency":"USD",
  "delayedPrice":null,
  "delayedPriceTime":null,
  "extendedChange":null,
  "extendedChangePercent":null,
  "extendedPrice":null,
  "extendedPriceTime":null,
  "high":null,
  "highSource":"IEX real time price",
  "highTime":1699626600947,
  "iexAskPrice":460.87,
  "iexAskSize":123,
  "iexBidPrice":435,
  "iexBidSize":100,
  "iexClose":436.61,
  "iexCloseTime":1699626704609,
  "iexLastUpdated":1699626704609,
  "iexMarketPercent":0.00864679844447232,
  "iexOpen":437.37,
  "iexOpenTime":1699626600859,
  "iexRealtimePrice":436.61,
  "iexRealtimeSize":5,
  "iexVolume":965,
  "lastTradeTime":1699626704609,
  "latestPrice":436.61,
  "latestSource":"IEX real time price",
  "latestTime":"9:31:44 AM",
  "latestUpdate":1699626704609,
  "latestVolume":null,
  "low":null,
  "lowSource":"IEX real time price",
  "lowTime":1699626634509,
  "marketCap":192892118443,
```

```
"oddLotDelayedPrice":null,  
"oddLotDelayedPriceTime":null,  
"open":null,  
"openTime":null,  
"openSource":"official",  
"peRatio":43.57,  
"previousClose":435.15,  
"previousVolume":2735507,  
"primaryExchange":"NASDAQ",  
"symbol":"NFLX",  
"volume":null,  
"week52High":485,  
"week52Low":271.56,  
"ytdChange":0.4790450244167119,  
"isUSMarketOpen":true  
}
```

توجه کنید که بین آکولادها، یک لیست از جفت‌های key-value با کاما از هم جدا شده‌اند، و هر کلید با یک علامت دو نقطه از مقدار خود جدا شده است. ما قصد داریم کاری مشابه با API پایگاه داده سهام خود انجام دهیم.

cs50.dev را باز کنید و `cd` را به تنهایی اجرا کنید. ترمینال شما باید شبیه نمونه زیر باشد:

```
$
```

دستور زیر را اجرا کنید تا فایل ZIP به اسم `finance.zip` را دانلود کنید:

```
wget https://cdn.cs50.net/2024/fall/psets/9/finance.zip
```

سپس دستور زیر را اجرا کنید تا فولدري به اسم `finance` بسازید:

```
unzip finance.zip
```

از اینجا به بعد دیگر نیازی به فایل ZIP ندارید پس با دستور زیر همراه با y آن را پاک کنید:

```
rm finance.zip
```

حالا با دستور زیر به فولدر مورد نظر منتقل شوید:

```
cd finance
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
finance/ $
```

در آخر با دستور ls باید بتوانید فایل‌ها و فولدرهای زیر را مشاهده کنید:

```
app.py  finance.db  helpers.py  requirements.txt  static/  templates/
```

اگر به مشکلی برخوردید، برگردید و ببینید که کدام مرحله را درست اجرا نکرده‌اید و مشکل کجاست!

وب سرور داخلی Flask را راه‌اندازی کنید. (finance/)

```
$ flask run
```

برای مشاهده عملکرد، از آدرسی که توسط Flask ارائه می‌شود استفاده کنید. با این حال، هنوز نمی‌توانید وارد سیستم شوید یا ثبت نام کنید!

در مسیر `finance/`، دستور `sqlite3 finance.db` را برای باز کردن پایگاه‌داده `finance.db` توسط `sqlite3` اجرا کنید. اگر دستور `schema` را در این محیط وارد کنید، خواهید دید که این پایگاه‌داده همراه با یک جدول به نام `users` ارائه شده است. نگاهی به ساختار این جدول کنید. دقت کنید که چطور به صورت پیش فرض کاربران `$10000` دریافت می‌کنند. اما اگر دستور `SELECT * FROM users;` را اجرا کنید، کاربری را مشاهده نخواهید کرد!

راه دیگری برای مشاهده `finance.db` استفاده از برنامه‌ای به نام `phpLiteAdmin` است. بر روی `finance.db` کلیک کنید، سپس بر روی لینک زیر متن “Please visit the following link to authorize GitHub Preview” کلیک کنید. باید بتوانید اطلاعاتی درمورد پایگاه‌داده و جدول `users` ببینید.

app.py

فایل app.py را باز کنید. بالای فایل، چند import به همراه ماژول SQL of CS50 و تعدادی تابع کمکی وجود دارد.

بعد از پیکربندی Flask، توجه کنید که این فایل حافظه‌ی کش پاسخ‌ها را غیرفعال می‌کند (به شرطی که در حالت debugging باشید، که به‌طور پیش‌فرض در فضای کد Code50 شما فعال است)، تا در صورتی که تغییری در فایلی ایجاد کنید، مرورگر شما آن را تشخیص دهد. توجه کنید که این فایل، Jinja را با یک فیلتر سفارشی به نام `usd` پیکربندی می‌کند، که یک تابع (تعریف‌شده در `helpers.py`) است و قالب‌بندی مقادیر به عنوان دلار آمریکا (USD) را آسان‌تر می‌کند. سپس Flask را طوری پیکربندی می‌کند که `sessions` را در سیستم فایل محلی (یعنی روی دیسک) ذخیره کند، به‌جای اینکه آن‌ها را داخل کوکی‌ها (دارای امضای دیجیتالی) نگه دارد، که تنظیم پیش‌فرض Flask است. سپس فایل ماژول SQL of CS50 را پیکربندی می‌کند تا از `finance.db` استفاده کند.

سپس مجموعه‌ای از مسیرها (routes) وجود دارد که فقط دو مورد از آن‌ها به‌طور کامل پیاده‌سازی شده‌اند: `login` و `logout`. ابتدا پیاده‌سازی `login` را بررسی کنید. توجه کنید که چگونه از `db.execute` (از کتابخانه CS50) برای کوئری گرفتن از پایگاه‌داده `finance.db` استفاده می‌کند. توجه کنید که چگونه از `check_password_hash` برای مقایسه هش‌های رمز عبور کاربران استفاده می‌شود. همچنین، دقت کنید که `login` با ذخیره `user_id` (یک مقدار INTEGER) در `session`، به خاطر می‌سپارد که یک کاربر وارد شده است. به این ترتیب، هریک از مسیرهای این فایل می‌تواند بررسی کند که در صورت وجود، کدام کاربر وارد سیستم شده است. در نهایت، توجه کنید که پس از ورود موفقیت‌آمیز کاربر، صفحه‌ی `login` کاربر را به صفحه‌ی اصلی "/" هدایت می‌کند. در عین حال، توجه کنید که `logout` به سادگی `session` را پاک می‌کند و به این ترتیب کاربر از سیستم خارج می‌شود.

توجه کنید که چطور بیشتر مسیرها با `@login_required` (تابعی که در `helpers.py` نیز تعریف شده است) دکوریت (decorate) شده‌اند. این دکوریتورها اطمینان می‌دهند که برای دسترسی به مسیرها، کاربر حتماً باید از قبل لاگین کرده باشد.

توجه کنید که اغلب مسیرها از هر دو متد GET و POST پشتیبانی می‌کنند. با این وجود، اکثر آنها فعلاً فقط `apology` را برمی‌گردانند زیرا هنوز پیاده‌سازی نشده‌اند!

helpers.py

بیا ببینیم به `helpers.py` نگاهی کنیم. اینجا نیز `apology` پیاده شده است. توجه کنید که چطور در نهایت یک قالب `apology.html` ارائه می‌دهد. در این تابع، تابع دیگری به نام `escape` تعریف شده است که از آن برای جایگزینی کارکترهای خاص در عذرخواهی‌ها استفاده می‌شود. با تعریف `escape` داخل `apology`، اولی را به دومی محدود کرده‌ایم؛ این به این معنی است که تابع دیگری قادر به فراخوانی آن نخواهد بود.

در ادامه، تابع `login_required` قرار دارد. ممکن است در نگاه اول کمی پیچیده به نظر برسد، اما نگران نباشید! اگر تا به حال از خود پرسیده‌اید که چگونه یک تابع می‌تواند تابع دیگری را برگرداند، در این یک مثال است!

بعد از آن `lookup` قرار دارد که یک `symbol` مثل `NFLX` را دریافت می‌کند و ارزش سهام یک شرکت را در قالب یک `dict` با سه کلید برمی‌گرداند:

۱. `Name`: مقدار آن یک رشته است.

۲. `Price`: مقدار آن یک `float` است.

۳. `Symbol`: مقدار آن یک رشته است که نسخه‌ای استاندارد (uppercase) از نماد سهام، بدون توجه به اینکه آن نماد چگونه هنگام وارد شدن به تابع `lookup` نوشته شده باشد است.

توجه داشته باشید که این قیمت‌ها "بلادرنگ" نیستند، و همانطور که در دنیای واقعی اتفاق می‌افتد، با گذشت زمان تغییر می‌کنند!

در آخر فایل یک تابع کوتاه به نام `usd` قرار دارد که یک `float` را به فرمت دلار آمریکا تبدیل می‌کند. برای مثال `۱۲۳۴/۵۶`

به شکل `$1,234.56` فرمت می‌شود

`requirements.txt`

با یک نگاه سریع به `requirements.txt` می‌توانیم بفهمیم که این فایل کتابخانه‌هایی که این برنامه به آنها وابسته است را در خود نگه می‌دارد.

`/static`

نگاهی اجمالی به `/static` داشته باشید. داخل این فولدر `styles.css` وجود دارد که برخی از CSS‌های اولیه را در بر گرفته است. البته می‌توانید به صلاح خود آنها را تغییر دهید.

`/templates`

در `login.html` فقط یک فرم HTML است که با `Bootstrap` طراحی شده است. در `apology.html`، یک قالب برای عذرخواهی وجود دارد. به یاد داشته باشید که `apology` در `helpers.py` دو آرگومان دریافت می‌کند: `message` که به عنوان مقدار `bottom` به `render_template` ارسال می‌شود و به طور اختیاری `code` که به عنوان مقدار `top` به `render_template` ارسال می‌شود. در `apology.html` توجه کنید که این مقادیر چگونه در نهایت استفاده می‌شوند! و دلیل آن اینجا است!

آخرین فایل `layout.html` است. این فایل کمی بزرگ‌تر از حد معمول است، اما عمدتاً به این دلیل که یک "navbar" زیبا و سازگار با موبایل دارد که بر اساس `Bootstrap` ساخته شده است. توجه کنید که چگونه یک بلاک به نام `main` تعریف کرده است که در داخل آن قالب‌ها (از جمله `login.html` و `apology.html`) قرار می‌گیرند. همچنین شامل پشتیبانی از ویژگی `message flashing` در `Flask` است تا بتوانید پیام‌ها را از یک مسیر به مسیر دیگر ارسال کرده و برای کاربر نمایش دهید.

Register

- پیاده‌سازی `register` را طوری کامل کنید که کاربر از طریق یک فرم بتواند ثبت نام کند.
- مطمئن شوید که کاربر یک نام کاربری وارد کند. این فیلد به عنوان یک فیلد متنی پیاده شده است که `name` برابر با `username` است. اگر ورودی کاربر خالی یا از قبل موجود بود، یک `apology` نمایش دهید.
 - توجه داشته باشید که در صورت وارد کردن نام کاربری تکراری، `cs50.SQL.execute` یک `exception` از نوع `ValueError` ایجاد می‌کند؛ زیرا ما یک `UNIQUE INDEX` روی ستون `users.username` ایجاد کرده‌ایم. بنابراین، حتماً از `try` و `except` استفاده کنید تا تکراری بودن یا نبودن نام کاربری را بررسی کنید.
 - وارد کردن رمز عبور برای کاربر الزامی است. این فیلد به عنوان فیلد متنی پیاده شده که `name` آن برابر است با `password`. سپس همان فیلد بار دیگر با `name` برابر `confirmation` پیاده می‌شود. اگر ورودی خالی بود یا پسوردهای گرفته شده یکی نبودند، یک `apology` نمایش دهید.
 - ورودی کاربر را از طریق `POST` به `/register` ارسال کنید.
 - کاربر جدید را با استفاده از `INSERT` به `users` اضافه کنید. به جای ذخیره پسورد کاربر، هش شده‌ی آن را ذخیره کنید. پسورد را با کمک `generate_password_hash` هش کنید. برای این کار احتمالاً نیاز داشته باشید که یک قالب جدید (مثل `register.html`) ایجاد کنید که کاملاً شبیه `login.html` باشد.
 - پس از پیاده‌سازی صحیح `register`، شما باید بتوانید یک کاربر را ثبت نام کنید. با اطلاعات او لاگین کنید. همچنین باید بتوانید اطلاعات خود را از طریق `phpLiteAdmin` یا `sqlite3` ببینید.

Quote

Quote را طوری کامل کنید که به کاربر اجازه دهد تا قیمت فعلی سهام را جستجو کند.

- مطمئن شوید که کاربر نماد سهام را وارد کند، که به صورت یک فیلد متنی، جایی که `name` برابر است با `symbol`، پیاده‌سازی شده است.
- ورودی کاربر را از طریق `POST` به `/quote` ارسال کنید.
- احتمالاً بخواهید دو قالب جدید ایجاد کنید (مثلاً `quote.html` و `quoted.html`). هنگامی که یک کاربر از طریق `GET` به `/quote/` مراجعه می‌کند، یکی از این قالب‌ها را نمایش دهید که درون آن یک فرم `HTML` قرار دارد که به `/quote/` از طریق `POST` ارسال می‌شود. در پاسخ به یک درخواست `POST`، `quote` می‌تواند قالب دوم را نمایش دهد و یک یا چند مقدار از `lookup` را در آن جایگذاری کند.

Buy

`Buy` را به گونه‌ای کامل کنید که کاربر بتواند از طریق آن سهام بخرد.

- الزامی است که کاربر یک نماد سهام وارد کند، که به صورت یک فیلد متنی، جایی که `name` برابر با `symbol` باشد، پیاده‌سازی شده است. در صورتی که ورودی خالی باشد یا نماد وجود نداشته باشد (مطابق مقدار بازگشتی از `lookup`)، یک پیام عذرخواهی نمایش دهید.
- الزامی است که کاربر تعداد سهام را وارد کند، که به صورت یک فیلد متنی، جایی که `name` برابر با `shares` باشد، پیاده‌سازی شده است. در صورتی که ورودی یک عدد صحیح مثبت نباشد، یک پیام عذرخواهی نمایش دهید.

- ورودی کاربر را از طریق POST به buy/ارسال کنید.
 - پس از تکمیل، کاربر را به صفحه اصلی هدایت کنید.
 - می‌توانید با فراخوانی lookup قیمت فعلی سهام را بررسی کنید.
 - می‌توانید با SELECT انتخاب کنید که کاربر چقدر پول نقد در users دارد.
 - یک یا چند جدول جدید به finance.db اضافه کنید تا خریدها را پیگیری کنید. اطلاعات کافی را ذخیره کنید تا مشخص شود چه کسی، چه چیزی را، با چه قیمتی و در چه زمانی خریداری کرده است.
 - از دیتا تایپ‌های مناسب در SQLite استفاده کنید.
 - ایندکس‌های UNIQUE را بر هر فیلدی که نیاز بود یکتا باشد تعریف کنید.
 - روی هر فیلدی که قرار است جستجو شود (مثلاً با استفاده از SELECT همراه با WHERE)، ایندکس‌های (غیر UNIQUE) تعریف کنید.
 - اگر کاربر قادر به خرید تعداد سهام با قیمت فعلی نیست، بدون انجام خرید یک apology نشان دهید.
 - لازم نیست نگران شرایط رقابتی باشید (یا از تراکنش‌ها استفاده کنید).
- بعد از پیاده‌سازی صحیح buy، باید بتوانید خریدهای کاربر را در جدول یا جداول جدی از طریق phpLiteAdmin یا sqlite3 ببینید.

index

پیاده‌سازی `index` را طوری کامل کنید که یک جدول HTML که شامل اطلاعات خلاصه‌ای است را برای کاربری که در حال حاضر وارد شده، نمایش دهد: سهام‌هایی که کاربر در اختیار دارد، تعداد سهام‌های متعلق به او، قیمت فعلی هر سهام و ارزش کل هر دارایی. همچنین باید موجودی نقدی فعلی کاربر را به همراه مجموع کل (به طور مثال ارزش کل سهام به علاوه پول نقد) نشان دهد.

- اگر بخواهید چند `SELECT` را اجرا کنید، بسته به اینکه چطور جداول خود را پیاده کرده‌اید، ممکن است `GROUP BY`، `HAVING`، `SUM` و یا `WHERE` در این فرایند به شما کمک کنند.
- می‌توانید از `lookup` برای هر سهم استفاده کنید.

Sell

`Sell` را بگونه‌ای کامل کنید که کاربر بتواند سهام‌هایی که در اختیار دارد را بفروشد.

- الزامی است که کاربر یک نماد سهام وارد کند، که به صورت یک منوی `select` جایی که `name` برابر با `symbol` پیاده‌سازی شده باشد. در صورتی که کاربر سهامی انتخاب نکند یا (به هر دلیلی، پس از ارسال) مالک هیچ سهمی از آن سهام نباشد، یک پیام عذرخواهی نمایش دهید.
- الزامی است که کاربر تعداد سهام را وارد کند، که به صورت یک فیلد متنی جایی که `name` برابر با `shares` باشد، پیاده‌سازی شده باشد. در صورتی که مقدار ورودی یک عدد صحیح مثبت نباشد یا کاربر به آن تعداد سهام از آن نماد سهام مالکیت نداشته باشد، یک پیام عذرخواهی نمایش دهید.

- ورودی کاربر را از طریق POST به sell/ارسال کنید.
- لازم نیست نگران شرایط رقابتی باشید (یا از تراکنش‌ها استفاده کنید).

History

- پیاده‌سازی history را بگونه‌ای کامل کنید که یک جدول HTML که شامل اطلاعات خلاصه شده‌ای از تمام معاملات کاربر تاکنون و فهرست بندی ردیف به ردیف هر خرید و فروش است را نشان دهد.
- برای هر ردیف، مشخص کنید که آیا یک سهام خریداری شده یا فروخته شده است و شامل نماد سهام، قیمت (خرید یا فروش)، تعداد سهام خریداری شده یا فروخته شده و تاریخ و زمان انجام تراکنش باشد.
 - ممکن است نیاز داشته باشید که جدولی را که برای خرید ایجاد کرده‌اید تغییر دهید یا آن را با یک جدول اضافی تکمیل کنید. سعی کنید از تکرارها جلوگیری کنید.

شخصی‌سازی

به انتخاب خود یک گزینه را برای شخصی‌سازی پروژه انتخاب کنید:

- به کاربران اجازه دهید پسوردهای خود را تغییر دهند.
- به کاربران اجازه دهید پول اضافی به حساب خود اضافه کنند.
- به کاربران اجازه دهید که بدون نیاز به تایپ نمادهای سهام به صورت دستی، سهام بیشتری بخرند یا سهم‌هایی که از قبل در اختیار دارند از طریق index بفروشند.
- پسوردهای کاربران را ملزم به داشتن تعدادی حروف، اعداد یا علامت کنید.
- برخی از ویژگی‌های محدوده قابل مقایسه را پیاده‌سازی کنید.

حتما وب اپلیکیشن خود را به صورت دستی تست کنید.

- ثبت نام یک کاربر جدید و بررسی اینکه صفحه پورتفولیوی آنها با اطلاعات صحیح بارگذاری می‌شود؛
 - درخواست قیمت با استفاده از نماد معتبر سهام؛
 - خرید چندباره یک سهم و بررسی اینکه پورتفولیو مجموع صحیح را نمایش می‌دهد؛
 - فروش تمام یا بخشی از یک سهم و بررسی دوباره پورتفولیو؛
 - بررسی اینکه صفحه تاریخچه‌ی شما تمام تراکنش‌های کاربر وارد شده را نمایش می‌دهد.
- همچنین برخی استفاده‌های غیرمنتظره را نیز تست کنید:
- وارد کردن رشته‌های الفبایی در فرم‌ها هنگامی که فقط اعداد مورد انتظار است؛
 - وارد کردن اعداد صفر یا منفی در فرم‌ها هنگامی که فقط اعداد مثبت مورد انتظار است؛
 - وارد کردن مقادیر float در فرم‌ها هنگامی که فقط اعداد صحیح مورد انتظار است؛
 - تلاش برای خرج کردن پول نقدی که بیشتر از دارایی کاربر است؛
 - تلاش برای فروش سهامی که بیشتر از مقدار دارایی کاربر است؛
 - وارد کردن نماد سهام نامعتبر؛
 - وارد کردن کاراکترهایی که به طور بالقوه در کوئری‌های SQL خطرناک هستند، مثل | و .

همچنین می‌توانید اعتبار HTML خود را با کلیک بر روی دکمه **I ♥ VALIDATOR** در پایین هر صفحه بسنجید. با این کار HTML شما به validator.w3.org POST می‌شود.

برای تست کد با `check50`، دستور زیر را اجرا کنید:

```
check50 cs50/problems/2025/x/finance
```

توجه داشته باشید که `check50` کل برنامه شما را به‌طور کلی تست می‌کند. اگر آن را قبل از تکمیل همه توابع مورد نیاز اجرا کنید، ممکن است برای توابعی که در واقع درست هستند اما به توابع دیگر وابسته‌اند، خطا گزارش دهد.

نحوه ارسال

```
submit50 cs50/problems/2025/x/finance
```

CS50x Iran

Harvard's Computer Science 50x Iran

